

Keecas Quarto Example

Table of contents

How to use keecas in a Quarto document	1
Basic Symbolic Math with Units	2
Pipe Commands Demonstration	2
Different LaTeX Environments	2
Cases Environment	2
Equation Environment with Labels	2
Verification Function	3
Subs get ordered	4
Check Function Templates	4
Custom Templates	4
Summary	5
Template Configuration	5

How to use keecas in a Quarto document

This example demonstrates all the key features of **keecas** for symbolic and units-aware calculations in a Quarto document.

i Note

If the rendering engine is **KaTeX** (i.e. Jupyter notebook), the label command `\label{}` will result in a **ParseError**. The latex code will work when rendering by **quarto render**, but since you may want to see the result before rendering the document, an option to disable the `\label{}` command is provided.

When rendering with **quarto** you can have two config:

- if **qmd** files, then everything is fine
- if **ipynb** files, then pass the `--execute` flag to **quarto render** to rerender the

Basic Symbolic Math with Units

Define symbols and calculate basic engineering quantities:

$$F = 10.00 \text{ kilonewton} \quad (\text{applied force}) \quad (1)$$

$$A = 50.00 \text{ cm}^2 \quad (\text{cross-sectional area}) \quad (2)$$

$$\sigma = \frac{F}{A} = 2.00 \text{ MPa} \quad (\text{normal stress}) \quad (3)$$

Pipe Commands Demonstration

Using pipe operators for functional composition:

$$x3.00 \text{ m} \quad (4)$$

$$y4.00 \text{ m} \quad (5)$$

$$d = x^2 + y^2 = 25.0 \text{ m}^2 \quad (6)$$

Different LaTeX Environments

Cases Environment

$$\begin{cases} E_{steel} = 200.00 \text{ gigapascal} & \text{steel elastic modulus} \\ E_{concrete} = 30.00 \text{ gigapascal} & \text{concrete elastic modulus} \end{cases} \quad (7)$$

Equation Environment with Labels

Calculate beam deflection with cross-references:

$$\delta = \frac{5 q L^4}{384 E I} = 15.949 \text{ mm} \quad (8)$$

i Note

labels emitted by `show_eqn` are latex labels, therefore they need to be referenced with `\eqref{}` or `\ref{}` latex commands.

`?@eq-QUARTO_EXAMPLE-delta` will not work

The deflection calculated in 8 shows acceptable values for serviceability.

Verification Function

Using the `check` function for design checks:

$$\sigma_{Sd} 20.00 \text{ MPa} \quad (9)$$

$$\tau_{Sd} 15.00 \text{ MPa} \quad (10)$$

$$\sigma_{Rd} 250.00 \text{ MPa} \quad (11)$$

$$\tau_{Rd} 75.00 \text{ MPa} \quad (12)$$

$$\frac{\sigma_{Sd}}{\sigma_{Rd}} = 0.080 \quad [\leq 1.0 \text{ VERIFIED}] \quad (13)$$

$$\frac{\tau_{Sd}}{\tau_{Rd}} = 0.200 \quad [\leq 1.0 \text{ VERIFIED}] \quad (14)$$

The type of comparison in the `check` function can be specified, as well the value to be specified against.

$$a = 3 \quad [> 1 \text{ NOT VERIFIED}] \quad (15)$$

$$b = 4 \quad [> 2 \text{ VERIFIED}] \quad (16)$$

$$c = 5 \quad [\geq 3 \text{ NOT VERIFIED}] \quad (17)$$

$$d = 6 \quad [\geq 4 \text{ VERIFIED}] \quad (18)$$

$$f = 7 \quad [\neq 5 \text{ NOT VERIFIED}] \quad (19)$$

Subs get ordered

Substitution will get ordered and sorted automatically, so you can use a symbol before mapping to a value.

Check Function Templates

The `check` function supports customizable templates for different visual styles:

Default : $[\leq 1.0 \text{ VERIFIED}]$ (20)

Boxed : $\boxed{\leq 1.0 \checkmark \text{ VERIFIED}}$ (21)

Minimal : $\leq 1.0 \checkmark$ (22)

$a = 3$ (23)

$b = 4$ (24)

$d = \frac{\sqrt{a^2 + b^2}}{c} = 0.417$ (25)

$c = a b = 12.000$ (26)

Custom Templates

You can also use completely custom templates:

Pass : $\leq 1.0 \text{ PASS}$ (27)

Fail : $> 1.0 \text{ FAIL}$ (28)

Summary

This example demonstrated:

- Basic symbolic math with units
- Pipe command usage (see Section)
- Different LaTeX environments (align, cases, equation)
- Label and cross-reference functionality
- Dataframe operations
- Verification functions
- Complex symbolic manipulations

All calculations in Section show that keecas provides a powerful interface for engineering calculations in Quarto documents.

Template Configuration

Templates can also be configured globally via TOML config files:

```
[check_templates.template_sets.minimal]
success = '${symbol}{rhs} \,\textcolor{{green}}{{\checkmark}}}'
failure = '${symbol}{rhs} \,\textcolor{{red}}{{\times}}}'
```

This allows you to set project-wide or user-wide styling for all check functions.